SE Unit-1

Software Engineering: It is the systematic, disciplined, and quantifiable approach to the development, operation and maintenance of software. In simple words, it means applying engineering principles to software development so that software is reliable, efficient, maintainable, and cost-effective.

Software Process: is a set of activities performed to develop and maintain software.

Main activities of a software process:

1. Requirement specification
   - Understant customer needs
   - Define functional & non-function requirements
2. Design
   - Decide how the software will be structure
   - Design database, user-interface, architecture, modules, etc.
3. Implementation
   - Programmers write the actual code based on the design
4. Testing
   - Software is checked for errors and whether it meets requirements.
5. Deployment
   - The completed software is delivered and installed for users.
6. Maintenance
   - Software is modified after delivery to fix bugs, improve performance, or add features.

Software Product: is a software system developed to provide a particular service or solve a particular problem for users.

Types of software products

1. Generic products
   - Developed for the general market.
   - Example: MS Office, Photoshop.
2. Customized products
   - Developed for a particular customer or organization
   - Example: Software developed specifically for a bank or hospital.

Emergence of Software Engineering:

The concept of Soft. Engg. emerged because software systems became larger and more complex.

During early days of computing, programs were relatively small and usually written by individual programmers. As computers became more powerful, organizations started developing large software systems.

This created several problems:

- Projects were taking long time.
- Development cost increased.
- Software contained many errors.
- Projects exceeded their budgets.
- Software was difficult to maintain
- Requirements were not properly understood.
- Less skilled staff
- S/w inefficiency
- Low quality s/w build
- Challenge in debug
- Low productivity

These problems became known as the software Crisis.

Scanned with OKEN Scanner

The term software Engineering became prominent around the late 1960s, particularly after the 1968 NATO conference on software engineering, where systematic engineering approaches to software development were discussed.

Evolving role of S/w:

- Early Era (1950s - 1960s): Batch processing, custom programs for scientific calculations, limited distribution.
- Second Era (1960s - 1970s): Multi-user systems, real-time databases, commercial off-the-shelf software packages
- Third Era (1980s - 1990s): Distributed computing, personal computing boom, embedded systems, networking.
- Modern Era (2000s - Present): Cloud-native architectures, SaaS, ubiquitous mobile computing, microservices, and AI-driven autonomous systems.

Characteristics of S/w: (developed not manufactured)

- S/w developed engineered in classical sense, not in physical sense.
- Does not "wear out": Unlike hardware, software does not suffer from friction, oxidation, or physical degradation. Instead, it deteriorates due to environmental changes.
- Intangible: software cannot be physically touched.
- Easy to modify: S/w can generally be modified to add or change functionality.
- High complex: Large S/w systems may contain millions of lines of code and many interacting components.

Applications: S/w is used in almost every area of modern life.

- Education: online learning, LMS
- Banking: Online banking, ATMs
- Entertainment: Games, streaming platform
- Transportation: Navigation, traffic mgt.
- Science: Research & simulation
- Manufactury: Automation & robotics
- Healthcare: medical digonsis
- Business: ERP, accounting software
- Communication: Email, messaging apps.
- Defence: Simulation, Communication
- E-commerce: online shopping
- Govt: Digital public services.

Software Crisis: refers to the difficulties faced in developing large and complex software systems, especially during 1960s & 1970s.

Major Problems:

- Projects were completed late.
- Development costs were very high
- Software foequently contained bugs
- Requirements were poorly understood.
- Software was difficult to maintain
- Projects exceeded budgets
- Reliability was poor
- Productivity was low.

Scanned with OKEN Scanner

Causes:
- Increasing software complexity
- Increasing size of software projects
- Poor planning
- Lack of proper development methods
- Poor requirement analysis
- Lack of standards
- Rapidly changing requirements

Solution: S/w engineering introduced systematic methods such as
- proper requirements analysis
- Structured design
- Software testing
- Project management
- Documentation
- Quality assurance
- Software life cycle models.

S/w Life Cycle Models:

Software development life cycle (SDLC) models defines how different software development activities are organized and performed.
Important life cycle models are:

1). Waterfall model: is a linear and sequencial s/w development model. Each phase is completed before moving to the next phase.

Requirements → System Design → Implementation → Testing → Deployment → maintenance

1. Requirements: Collect and document all requirements.
2. Design: Design the architecture, database, interfaces, etc.
3. Implementation: Write the program / code.
4. Testing: Test the complete software and find errors.
5. Deployment: Deliver the software to users.
6. Maintenance: Fix errors and make necessary changes.

- Use this model when:
  - Requirements are clear & fixed
  - The project is relatively stable
  - Technology is well understood.
  - Changes are unlikely.
- Example: Govt. or regulatory project where requirements are clearly documented beforehand.

- Advantages:
  - Simple and easy to understand
  - Easy to manage because phases are clearly defined
  - Proper documentation is maintained
  - Easy to track progress
  - Suitable when requirements are stable.

- Disadvantages:
  - Difficult to handle changing requirements
  - Working software is available only relatively late.
  - Errors in requirements may be discovered late
  - Testing happens after implementation
  - Not suitable for highly uncertain projects.

Scanned with OKEN Scanner

2). Incremental Model: In this model, the complete software is divided into small functional parts called increments. Each increment is developed and delivert separately.

Increment 1 → Basic features
↓
Increment 2 → More features
↓
Icrement 3 → More features
↓
Final Product.

Example: Suppose you are developing an online shopping system.

Increment 1: • Login • Registration
Increment 2: • Product search
• Product details
Increment 3: • Cart • Payment
Increment 4: • Order tracking

The customer gets a working system early, and functionality is added gradually.

When to use?

- The software can be divided into independent features.
- Early ~~dt~~ delivery is important
- Requirements may change.
- The customer wants usable software quickly.

Adv.

- Working software is delivered early
- Easier to handle changing requirements
- Customer feedback can be incorporated.
- Easier testing and debugging of smaller parts.
- Lower risk compared with delivering everything at once.

Dis. Adv.

- Requires good planning and architecture.
- Integration blw increments can be difficult.
- Total cost may increase if increments are poorly managed
- Not every system can be easily divided into independent increments.

3) RAD Model: (Rapid Application Development)

The RAD model focuses on quick development and rapid delivery using reusable components, prototyping, user feedback, and development tools.
- generally emphasizes short development cycles and strong user involvement.

1. Business Modelling: Understand the organization's business requirements and information flow.
2. Data Modelling: Identify data required by the system and relationship blw data
3. Process modelling: Define how the data will be processed.
4. Application Generation: Develop the application using reusable components and development tools.
5. Testing & Turnover: Test the system and deliver ~~to~~ it to users.

Business Modelling
↓
Data Modelling
↓
Process modelling
↓
Application Generation
↓
Testing & Turnover

Scanned with OKEN Scanner

useful when.

- Quick delivery is required
- Requirements are reasonably well understood.
- The project can be divided into modules
- Reusable components / tools are available.
- developers and users can work closely together.

Adv.

- Faster development.
- Early delivery of software
- Frequent user feedback
- Reusable components reduce development effort
- Changes can be incorporated relatively easily.

Dis adv.

- Require skilled developers.
- Require continuous customer / user involvement.
- Not suitable for very large / highly complex systems.
- Requires sufficient resources
- Not suitable when the system cannot be divided into modules.

4). Prototype Model: creates an early working model (prototype) of the software to understand and refine user requirements.

- It is especially useful when the customer is not sure exactly what they want

Example:
Suppose a customer wants a new mobile application but cannot clearly explain how interface should look.

1. Developer collects basic requirements.
2. A simple prototype is created.
3. Customer sees and uses it.
4. Customer gives feedback.
5. Prototype is modified.
6. This process continues until requirements are clear.
7. Final software is developed.

Initial Requirements
↓
Quick Design
↓
Prototype
↓
User Evaluation
↓
Improved Prototype
↻
↓
Final Requirements
↓
Final System.

- When to use?

- Requirements are unclear.
- customer needs are difficult to understand
- User interface is important.
- Frequent customer feedback is required.

Adv.

- Helps identify requirement clearly
- Customer gets an early view of system
- Reduces misunderstanding b/w developer and customer.
- Useful for UI/UX development.
- Changes can be identified early.

Disadv.

- Customer may think the prototype is the final product.
- Frequent changes can increase cost & t
- Poorly designed prototypes may lead poor architecture
- Too much customer involvement may required.

Scanned with OKEN Scanner

5) Agile Process Model: Agile is an iterative and incremental approach in which software is developed in small cycles with continuous customer feedback.

- Instead of developing the whole system at once, teams develop small features, test them, get feedback, and improve the product continuously.
- Each iteration is usually called a sprint in scrum.

Example: For a food-delivery application:

Sprint 1: login + Registration
↓
Sprint 2: Restaurant listing
↓
Sprint 3: Cart + ordering
↓
Sprint 4: Payment
↓
Sprint 5: Order tracking

- Each sprint produces a working improvement.

Planning
↓
Design
↓
Code
↓
Test
↓
Release
↓
Customer feedback
↓
Next iteration.

Important Principles

- Customer collaboration
- Frequent delivery of working software
- Team collaboration
- Continuous feedback
- Responding to changing requirements
- Continuous testing & improvement.

When to use?

- Requirements change frequently
- Customer feedback is important
- Fast delivery is required
- The project is complex or uncertain
- The development team can communicate frequently with stakeholders.

Adv.

- Easily accommodate changing req's
- Frequent delivery of working s/w.
- Continuous customer feedback
- Problems can be identified easily.
- Continuous testing improves quality
- Good communication within the team.

Disadv.

- Requires active customer/stakeholder involvement.
- Requires experienced & collaborative teams.
- Difficult to predict exact cost and schedule when req's continuously change.
- Documentation may receive less emphasis if the process is poorly managed

Agile testing methods (Agile software development methods)

1) Scrum: {work in short sprints}
   - Work is divided into short sprints (usually 1-4 weeks)
   - Daily meetings & regular reviews are used
   - Focuses on teamwork and continuous feedback

{→ Scrum master, → product owner, → Scrum team}

Scanned with OKEN Scanner

2) Crystal { focuses on people & communication }

- Focuses on people, communication, and teamwork.
- Method is adapted acc. to team size and project complexity
- Team member communicate frequently
- Allows changes during development.

3) DSDM (Dynamic System Development method). { Rapid development with user involvement }

- Focuses on rapid application development.
- Users are actively involved throughout development
- Important features are developed and delivered first
- Time & resources are generally fixed.

4) FDD (Frequent-driven development) { Develop feature by feature }

- Software is divided into small features.
- Features are listed and prioritized.
- Each feature is designed, coded and tested separately
- Completed features are gradually combined into the final system

5) Lean Software development { Remove waste and maximum value }

- Focuses on removing unnecessary work (waste).
- Delivers valuable software quickly.
- Avoids unnecessary features and repeated work.
- Encourages continuous improvement.
- Give importance to customer value.

6) XP (Extreme Programming) { Frequent development testing & feedback }

- Focuses on high-quality and frequent software delivery
- Development is done in small releases.
- Continuous testing is performed.
- Customer feedback is taken frequently.
- Easily adapts to changing req's.

Comparisons.

Feature	Waterfall	Incremental	RAD	Prototype	Agile
• Development	Sequential	in increments	Rapid	Prototype first	Iterative
• Requirements	fixed	can change	Fairly understood	Unclear	Frequently changing
• Customer involvement	low	medium	High	High	High
• Working software	Late	Early	Very early	Prototype early	Frequent
• Changes	Difficult	Easier	Easier	Very easy initially	Very easy
• Main focus	Planning	Feature-wise delivery	Speed	Requirement understanding	Flexibility
• Best suited for	Stable projects	Modular system	Fast	Unclear requirements	Dynamic projects

Scanned with OKEN Scanner

Need of Software Engineering: SE is needed. We modern software is large, complex and critical.

1) Managing complexity: large sw need systematic development methods.
2) Reducing development cost
3) Improving quality
4) Meeting deadlines
5) Easy maintenance
6) Managing changing requirements
7) Risk management
8) Team coordination

Goal of SE:

1) Correct: should perform the functions specified by req's.
2) Reliable: Should work correctly and consistently.
3) Efficient: Should use resources such as CPU, memory, storage, and network efficiently.
4) Maintainable: should be easy to modify, fix and improve
5) Usable: should be easy to learn and operate.
6) Secure: Should protect data & prevent unauthorized access.
7) Reusable: Components should be reusable wherever possible.
8) Scalable: System should be able to handle inc. users, data or workload.
9) Cost effective: Should be developed & maintained within reasonable cost.
10) Delivered on time: project should meet its planned schedule.

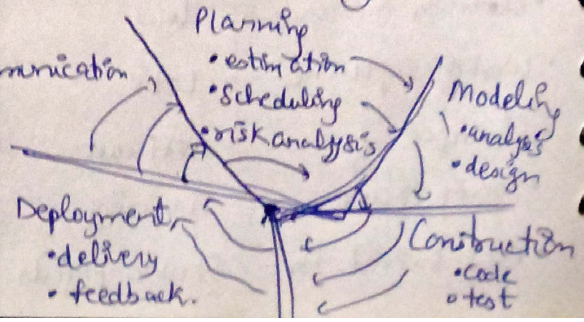
Spiral model: is a software development model that combines iterative development with risk analysis. The software is developed through repeated cycles called Spirals.

- Each cycle produces an improved version of software
- Barry Boehm [Bo 688]

when to use?

- Large & complex projects.
- Projects with high risks.
- Requirements are likely to change
- When continuous customer feedback is needed.

Planning
↓
Risk analysis
↓
Development & Testing
↓
Customer Evaluation
↓
Next Spiral



Scanned with OKEN Scanner

Adv.

- Strong focus on risk management
- Changes can be incorporated easily
- Customer feedback is obtained
- Problems can be identified early.
-

Disadv.

- Expensive and complex
- Requires skilled risk analysis
- Not suitable for small/simple projects
- Management can be difficult.

Software Project Management

- SPM is the process of planning, organizing, monitoring and controlling the resources and activities required to develop and deliver software successfully.
- main objective:
  - → Within the planned time
  - → Within the budget
  - → Using available resources effectively.
  - → With the required quality
  - → Acc. to customer's requirements
- Main activities:

1) project planning: deciding what needs to be done
2) Project estimation: estimating effort, cost, time and resources.
3) Scheduling: deciding when each task will be performed.
4) Resource allocation: assigning people, tools and other resources
5) Risk management: identifying and handling possible problems.
6) Team management: selecting and coordinating the project team.
7) Project tracking: check whether the project is progressing acc to plan
8) Quality management: ensuring the software meets required standards.
9) Reporting and communication: communicating progress to stakeholders.
10) Project delivery and maintenance: delivering the completed product and supporting it afterward.

Software Process: is the set of activities/steps followed to develop and maintain software.

Example:
requirement analysis, design, coding, testing, Deployment, maintenance.

Software Process metrics: Measures the software development process

Example:

- Defect removal efficiency
- Average time to fix defects
- Development productivity
- Testing efficiency

They help an organisation improve its development process.

Scanned with OKEN Scanner

Project metrics: Measures the progress and performance of a particular project

Examples:

- Cost
- Schedule
- Effort
- No. of completed tasks
- Number of unresolved issues
- Resource utilization.

Project Planning:

- Project planning is the process of deciding what work has to be done, how it will be done, who will do it, when it will be done and how much it will cost.
- good project plan acts as a roadmap for the project.

Steps in project planning:

1) Define project objectives : clearly determine what the project is supposed to achieve
2) Define project scope : determine what is included and excluded from the project.
3) Identify deliverables : deliverables are the outputs that must be produced.
4) Identify tasks : break project into smaller activities.
5) Estimate effort, cost and resources
6) Prepare schedule
7) Allocate resources
8) Identify risks
9) prepare monitoring & control plan.

Project Estimation: means predicting the effort, cost, duration and resources required to complete a software project.

factors affecting estimation:

- software size
- Complexity
- Developer experience
- Techology used
- Required quality
- Available resources
- Project duration
- Risk
- Customer requirement.

Project Estimation Techniques:

1) Expert Judgement: Experienced people estimate the project based on their knowledge and previous experience.
2) Analogy-Based Estimation: Current project is compared with a similar completed project.
3) Decomposition Technique: Complete project is divided into smaller tasks and each task is estimated separately.
   This is often called bottom-up estimation.

Scanned with OKEN Scanner

4) Algorithmic / Parametric Estimation: A mathematical formula is used to calculate effort or cost.
Example: COCOMO.

Empirical Estimation Techniques: uses historical project data and mathematical relationships to estimate a new project.

COCOMO Model: (Constructive Cost Model)

- It was proposed by Barry Boehm.
- It is used to estimate: development effort, time and approximate staffing requirements.

The input is mainly the estimated software size in KLOC.
KLOC = Kilo Lines of Code = thousand lines of code.

- Classify projects in three categories.

1). Organic Mode

Used for:
- Small/Simple projects
- Familiar environment
- Experienced team
- Relatively simple req's

Ex: Inventory management system

2) Semi-detached Mode

Used for:
- Medium sized project
- Moderate complexity
- Team may contain people with diff. levels of experience.

Ex: OS or DBMS-type project

3). Embedded Mode

Used for
- Large & complex projects
- Strong hardware/s/w constraints
- Strict req's.

Ex: Air traffic cont
Medical control s

Basic COCOMO model:

- Effort $E = a(KLOC)^b$ pm
- Development Time $T_{dev} = c(E)^d$

~~KLOC = abcd~~

- E = effort in person-months
- a, b = constants
- T_{dev} = development time in mon
- c, d = constants.

Estimation of Development Effort:

Organic: $Effort = 2.4(KLOC)^{1.05}$ PM
Semi-detached: $E = 3.0(KLOC)^{1.12}$ PM
Embedded: $E = 3.6(KLOC)^{1.20}$ PM

Estimation of Development Time:

Organic: $T_{dev} = 2.5(Effort)^{0.38}$ months
Semi-detached: $T_{dev} = 2.5(E)^{0.35}$ months
Embedded: $T_{dev} = 2.5(E)^{0.32}$ months.

Scanned with OKEN Scanner

Staffing Level Estimation: Staffing level means determining how many people are needed and when they are needed during the project.

- The no. of people is not necessarily constant throughout the project.

Avg. Staff = $\frac{\text{Total Effort}}{\text{Development Time}}$

Risk: is a possible future event that can negatively affect a project.

Risk management:

1) Risk Identification: Find possible risks.
   Example: requirement changes, staff leaving, technology failure, budget shortage, security problems, Schedule delay, hardware failure.

2) Risk Analysis:
   - Determine how likely is the risk? How serious will its impact be?
   - Simple approach -
     Risk Exposure = Probability × Impact.

3) Risk Prioritization: risk with high probability & high impact should receive more attention.

4) Risk response Planning: decide what to do about each risk.
   Avoid, mitigate, Transfer, Accept.

5) Risk Monitoring: continuously monitor identified risks and look for new ones.

- Categories: Project risks, Technical risks, Business risks.

Project Scheduling: determines when project activities will start & finish and in what order they should be performed.

- It converts project plan into a time-based schedule.

Steps.

1) Identify project activity
2) Estimate duration of each activity
3) Identify dependencies
4) Arrange activities in order
5) Assign resources
6) Set start & finish date
7) Identify milestones
8) Monitor and update the schedule.

Scanned with OKEN Scanner

Project Tracking: means continuously comparing actual project progress with the planned schedule, effort and cost.

Cycle:

Plan → Execute → Measure actual progress → Compare with plan → Find deviation → Take corrective action → update plan.

Schedule & Cost metrics:

- **Schedule metrics**

1) SPI (Schedule Performance Index): measures schedule efficiency.

$$SPI = \frac{EV}{PV}$$ where EV = Earned value, PV = Planned value

SPI > 1 : ahead of schedule
SPI = 1 : on schedule
SPI < 1 : behind schedule.

2) SV (Schedule Variance): $SV = EV - PV$

- SV > 0 : ahead of schedule
- SV = 0 : on schedule
- SV < 0 : behind schedule.

3) PV (Planned value): budgeted value of work that was planned to be completed by a particular time.

- **Cost metrics**

4) EV (Earned Value): budgeted value of the work actually completed.

2) AC (Actual Cost): actual amt. of money spent on work completed.

3) CPI (Cost Performance Index): $CPI = \frac{EV}{AC}$

- CPI > 1 : spending is more efficient than planned
- CPI = 1 : exactly acc. to budget
- CPI < 1 : cost performance is worse than plan.

4) CV (Cost Variance): $CV = EV - AC$

- CV > 0 : under budget
- CV = 0 : on budget
- CV < 0 : over budget.

Milestone: is an imp. point or achievement in a project sched. It normally represents the completion of an imp. phase or deliverable.

Scanned with OKEN Scanner

Types of Software team:

1) Generalist team: consists of members who have multiple skills and can perform different types of tasks.
   - Team members can handle more than one role.
   - Work is shared among members.
   - Suitable for small projects & agile environment.
   - Communication & collaboration are imp.

2). Specialized Team: consists of members who are experts in specific areas.
   Each member performs a particular role.
   ex: UI/UX designer, Developer, Tester, Database expert.

3) Hybrid team: Combines generalized and specialized team.
   - Some member have broad skills, while others are specialists
   Ex: developes may handle coding & testing.

Scanned with OKEN Scanner